



WORKBOOK PESERTA & ASISTEN

Pelatihan Internet of Things

Fire Detector (Monitoring Kualitas Udara) & Smart Absensi berbasis ESP32

Kegiatan Pengabdian kepada Masyarakat (PkM)

Pelaksana: Dosen STMIK Tazkia (Endy Muhardin) + asisten mahasiswa

Mitra: Universitas Pancasila · Pendukung: PT ArtiVisi Intermedia

Juni 2026

Workbook ini dipakai oleh **peserta** maupun **asisten** — isinya sama. Tiap langkah ditulis lengkap dan bisa di-copy-paste. Catatan **"Kalau error"** ada inline di tiap langkah; kalau mentok, panggil trainer.

Daftar Isi

Cara Pakai Workbook	2
Konvensi penulisan	2
Soal token & password	2
Fundamental IoT & Setup	3
Apa itu IoT, dan kenapa ESP32	3
Breadboard	3
Install Arduino IDE + dukungan ESP32	3
Install library	4
Buat akun & device Blynk	4
Mengisi secrets.h	4
Latihan 1: Blink LED bawaan	5
Latihan 2: Konek WiFi + kirim 1 data ke Blynk	5
Lab Fire Detector	7
Tujuan & konsep	7
Daftar komponen (BOM) & estimasi biaya	7
Wiring pin-to-pin	8
Setup datastream Blynk	9
Firmware	9
Menjalankan & kalibrasi ambang	11
Lab Smart Absensi	12
Tujuan & konsep	12
Daftar komponen (BOM) & estimasi biaya	12
Wiring pin-to-pin	12
Setup datastream Blynk	14
Firmware	14
Menjalankan	16
Lampiran A — Troubleshooting umum	18
Lampiran B — Demo Smart Irrigation	19

Cara Pakai Workbook

Pelatihan ini berformat **peer-led**: asisten memandu lab, trainer mengawasi dan menangani eskalasi. Alurnya:

Setengah hari pertama	Fundamental IoT + setup tool (Bab 1). Semua peserta barengan, dipandu trainer.
Pembagian kelompok	4 kelompok (masing-masing 2 orang) dibagi ke 2 lab, berjalan konkuren : 2 kelompok Fire Detector, 2 kelompok Smart Absensi.
Hands-on	Tiap kelompok kerjakan lab-nya (Bab 2 atau Bab 3) dipandu asisten. Konsep universal (WiFi, Blynk) dijelaskan plenary singkat.
Penutup	Demo swap antar kelompok + demo Smart Irrigation (Lampiran B).

Konvensi penulisan

- Kotak abu-abu = kode untuk di-copy-paste persis.
- **Kalau error** = titik gagal umum + cara perbaiki, ditulis tepat di langkah yang rawan.
- **Penting** = hal yang kalau salah bisa **merusak komponen** (mis. tegangan 3.3V vs 5V). Baca pelan-pelan.

Soal token & password

Token Blynk dan password WiFi **tidak** ditulis di kode yang di-commit. Kode membaca file `secrets.h`.
Tiap folder firmware sudah berisi `secrets.h.example` — salin jadi `secrets.h`, lalu isi. Detailnya di bagian *Mengisi secrets.h*.

Fundamental IoT & Setup

Apa itu IoT, dan kenapa ESP32

Internet of Things (IoT) = menghubungkan alat fisik (sensor, lampu, motor) ke internet supaya datanya bisa dipantau dan dikontrol dari jarak jauh. Pola dasar tiap proyek IoT di pelatihan ini sama:

Sensor → Mikrokontroler → WiFi → Cloud → Smartphone

Mikrokontroler = komputer mungil satu chip yang menjalankan satu program. Kita pakai **ESP32 DevKit V1**: murah, sudah ada **WiFi** bawaan, dan punya banyak kaki (**pin**) untuk dicolok sensor.

- **GPIO** (General Purpose Input/Output) = pin serbaguna ESP32. Tiap pin punya nomor (mis. GPIO27). Lewat program, satu pin bisa jadi **input** (baca sensor) atau **output** (nyalakan buzzer/LED).
- **Pin analog (ADC)** = pin yang bisa baca tegangan bertingkat (bukan cuma on/off). ESP32 membacanya sebagai angka 0–4095. Dipakai untuk sensor seperti MQ-2 yang outputnya “seberapa banyak”, bukan sekadar ada/tidak.
- **Pin digital** = hanya baca/tulis dua kondisi: HIGH (~ 3.3V) atau LOW (0V).

Breadboard

Breadboard = papan untuk merangkai komponen tanpa menyolder. Lubang-lubangnya terhubung di dalam dengan pola:

- Dua jalur panjang di tepi (bertanda + dan –) = jalur **power**. Biasa dipakai untuk membagikan 3.3V/5V dan GND ke banyak komponen.
- Lubang di tengah terhubung **per kolom** (5 lubang vertikal jadi satu), terpisah oleh celah tengah.

PENTING

GND harus nyambung jadi satu (common ground). Semua GND komponen + GND ESP32 disatukan di jalur – breadboard. Kalau tidak, sensor baca nilai ngaco.

Install Arduino IDE + dukungan ESP32

1. Download **Arduino IDE 2.x** dari arduino.cc/en/software, install seperti aplikasi biasa.
2. Buka **File** → **Preferences**. Di kolom **Additional boards manager URLs**, tempel: https://espressif.github.io/arduino-esp32/package_esp32_index.json
3. Buka **Tools** → **Board** → **Boards Manager**, cari **esp32**, install paket “**esp32 by Espressif Systems**”.
4. Colok ESP32 ke USB. Pilih **Tools** → **Board** → **ESP32 Arduino** → “**ESP32 Dev Module**”.
5. Pilih **Tools** → **Port** → port yang muncul (mis. `/dev/cu.SLAB_USBtoUART` atau `COM3`).

KALAU ERROR

Port tidak muncul / ESP32 tidak terdeteksi. ESP32 butuh driver USB-serial. Cek chip di sebelah port USB board: **CP2102** → install driver Silicon Labs CP210x; **CH340** → install driver CH340. Setelah install, cabut-colok ulang kabel. Pastikan kabel USB-nya kabel **data**, bukan kabel charge-only.

Install library

Buka **Sketch** → **Include Library** → **Manage Libraries**, lalu install (ketik nama, klik Install, terima kalau diminta install dependency):

Library	Dipakai di
Blynk (by Volodymyr Shymanskyi)	Semua — kirim data ke cloud
DHT sensor library (Adafruit) + Adafruit Unified Sensor	Fire Detector
MFRC522 (by GithubCommunity)	Smart Absensi — baca RFID
Adafruit SSD1306 + Adafruit GFX Library	Smart Absensi — OLED

Buat akun & device Blynk

Blynk = layanan cloud + app smartphone untuk dashboard IoT. Data dari ESP32 dikirim ke **Virtual Pin** (V0, V1, ...) lalu ditampilkan sebagai widget.

1. Daftar di blynk.cloud (gratis untuk pemakaian kecil).
2. **Developer Zone** → **Templates** → **New Template**. Beri nama (mis. "Fire Detector"), Hardware = **ESP32**, Connection = **WiFi**.
3. Tab **Datastreams** → **New Datastream** → **Virtual Pin**. Buat pin sesuai tabel di tiap lab (V0, V1, ...). Set tipe data (Integer/Double/String) dan range sesuai tabel.
4. Tab **Web Dashboard**: tarik widget (Gauge, Label, LED) dan hubungkan ke datastream tadi. Ini yang Anda lihat saat alat jalan.
5. Menu **Devices** → **New Device** → **From template** → pilih template tadi. Buka device, **Device Info** berisi `BLYNK_TEMPLATE_ID`, `BLYNK_TEMPLATE_NAME`, dan `BLYNK_AUTH_TOKEN`.
6. Install app **Blynk IoT** di HP, login akun sama, untuk lihat dashboard dari HP.

Mengisi secrets.h

Tiap folder firmware berisi `secrets.h.example`. Salin jadi `secrets.h` (di folder yang sama), buka di Arduino IDE, isi nilainya:

```
#define BLYNK_TEMPLATE_ID   "TMPLxxxxxxxx"
#define BLYNK_TEMPLATE_NAME "Fire Detector"
#define BLYNK_AUTH_TOKEN    "isi-token-device-dari-blynk"
#define WIFI_SSID           "nama-wifi"
#define WIFI_PASS            "password-wifi"
```

PENTING

`secrets.h` sudah masuk `.gitignore` supaya token & password tidak ikut ter-upload ke GitHub. Jangan menaruh token langsung di file `.ino`.

KALAU ERROR

WiFi kampus pakai halaman login (captive portal). ESP32 tidak bisa mengisi form login web. Solusi: pakai **hotspot HP** sebagai gantinya — isikan SSID & password hotspot di `secrets.h`.

Latihan 1: Blink LED bawaan

Sebelum menyentuh sensor, pastikan rantai “tuliskan kode → upload → jalan” sudah benar. ESP32 punya LED bawaan di GPIO2.

```
void setup() {  
  pinMode(2, OUTPUT);  
}  
void loop() {  
  digitalWrite(2, HIGH); // LED nyala  
  delay(500);  
  digitalWrite(2, LOW);  // LED mati  
  delay(500);  
}
```

Klik tombol **Upload** (panah kanan). Saat muncul “Connecting...”, kalau gagal, **tahan tombol BOOT** di ESP32 beberapa detik sampai upload mulai. LED bawaan akan kedip tiap 0.5 detik.

KALAU ERROR

Upload gagal / “Failed to connect / Timed out waiting for packet header”. Tahan tombol **BOOT** saat muncul “Connecting....”. Pastikan Port benar dan tidak ada Serial Monitor lain yang sedang membuka port itu.

Latihan 2: Konek WiFi + kirim 1 data ke Blynk

Ini fondasi kedua lab. Buat device Blynk (bagian *Buat akun & device Blynk*) dengan 1 datastream **V0** (Integer). Buat `secrets.h` di folder sketch ini. Kode:

```
#include "secrets.h"  
#include <WiFi.h>  
#include <BlynkSimpleEsp32.h>  
  
BlynkTimer timer;  
int hitung = 0;  
  
void kirim() {  
  hitung++;  
  Blynk.virtualWrite(V0, hitung); // kirim angka naik ke V0  
  Serial.print("kirim V0 = "); Serial.println(hitung);  
}  
  
void setup() {  
  Serial.begin(115200);  
  Blynk.begin(BLYNK_AUTH_TOKEN, WIFI_SSID, WIFI_PASS);  
  timer.setInterval(2000L, kirim);  
}  
  
void loop() {  
  Blynk.run();  
  timer.run();  
}
```

Buka **Tools** → **Serial Monitor**, set baud **115200**. Harusnya muncul log koneksi WiFi lalu “Ready”, lalu angka naik tiap 2 detik. Tambahkan widget **Label** ke V0 di dashboard Blynk — angkanya ikut naik.

KALAU ERROR

Serial Monitor isinya karakter aneh / kotak-kotak. Baud rate salah. Set ke **115200** di pojok kanan bawah Serial Monitor.

KALAU ERROR

Macet di “Connecting to wifi...” terus. SSID/password salah, atau WiFi captive portal (lihat catatan di bagian *Mengisi secrets.h*). ESP32 hanya support WiFi **2.4GHz**, bukan 5GHz.

KALAU ERROR

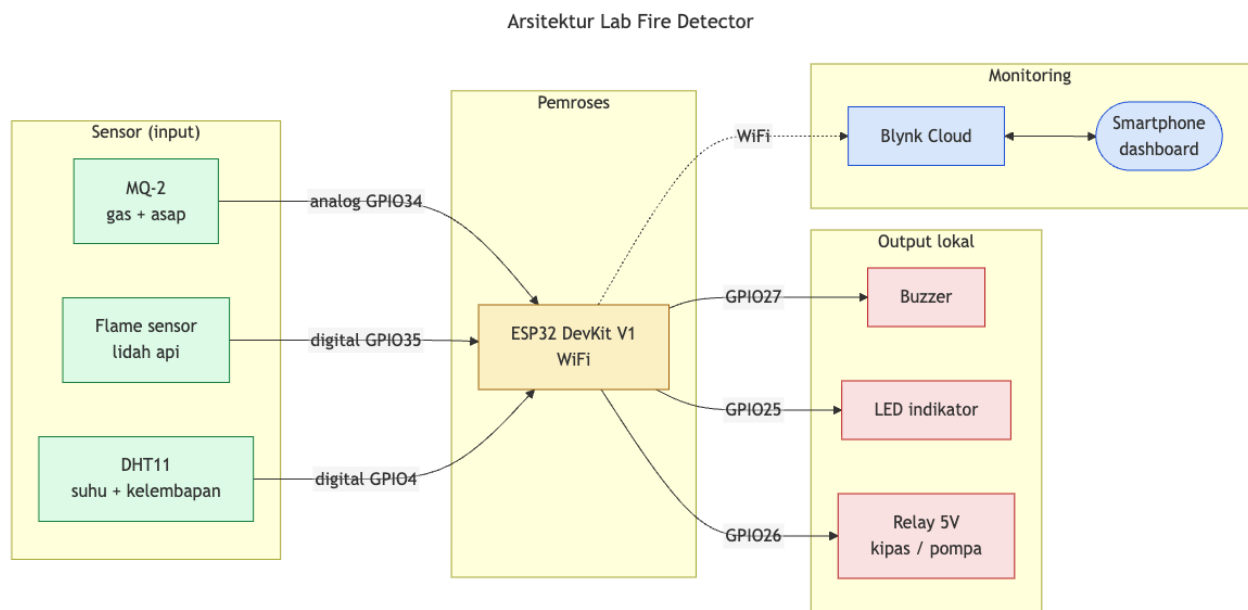
“Invalid auth token”. `BLYNK_AUTH_TOKEN` salah ketik atau token device lain. Salin ulang dari Blynk → Device → Device Info.

Lab Fire Detector

Tujuan & konsep

Membangun pendeteksi kebakaran + monitoring kualitas udara: baca **gas/asap** (MQ-2, sensor analog), **api** (flame sensor, digital), dan **suhu/kelembapan** (DHT11). Kalau gas melewati ambang ATAU api terdeteksi → bunyikan buzzer, nyalakan LED, aktifkan relay (mensimulasikan kipas exhaust / pompa), dan kirim status ke Blynk.

Konsep yang dilatih: **baca sensor analog (ADC) + ambang batas (threshold)**, aktuasi via relay, dan kirim telemetri ke cloud. Arsitektur sensor gas ini sama dengan use-case **monitoring kualitas udara** yang diminta — bedanya hanya ambang & interpretasi.



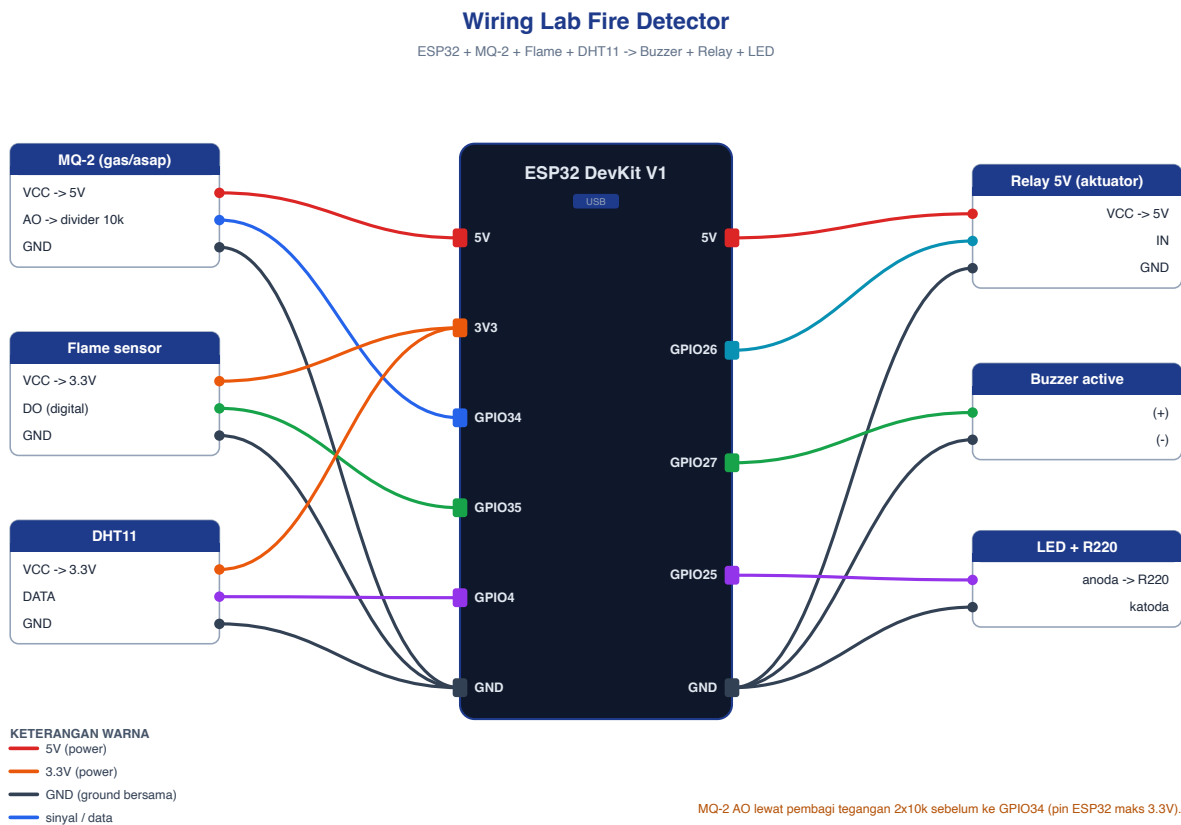
Daftar komponen (BOM) & estimasi biaya

No	Komponen	Fungsi	Estimasi
1	ESP32 DevKit V1	Mikrokontroler + WiFi	Rp 65.000
2	Sensor MQ-2	Deteksi gas LPG & asap (analog)	Rp 15.000
3	Flame sensor IR	Deteksi lidah api (digital)	Rp 8.000
4	DHT11	Suhu & kelembapan	Rp 20.000
5	Relay module 5V 1ch	Saklar aktuator (kipas/pompa)	Rp 12.000
6	Buzzer active	Alarm suara	Rp 5.000
7	LED + resistor 220Ω	Indikator visual	Rp 5.000
8	Resistor 10kΩ (2 pcs)	Pembagi tegangan MQ-2	Rp 2.000
9	Breadboard + kabel jumper	Perakitan tanpa solder	Rp 40.000
Total estimasi per kelompok			Rp 172.000

Wiring pin-to-pin

PENTING

Susun rangkaian saat ESP32 **tidak** terhubung USB/power. Colok power baru setelah semua kabel terpasang dan dicek ulang.



Warna kabel pada diagram = peran (merah 5V, oranye 3.3V, abu GND, biru sinyal). Tabel di bawah adalah rujukan pasti pin-per-pin.

MQ-2 (sensor gas, analog). MQ-2 perlu 5V untuk elemen pemanasnya. Tapi output AO bisa mendekati 5V saat asap pekat, padahal pin ESP32 maksimal **3.3V**. Karena itu AO dilewatkan **pembagi tegangan** 2 resistor 10kΩ dulu.

Pin MQ-2	Ke
VCC	5V (pin VIN / 5V ESP32)
GND	GND
AO	titik tengah pembagi tegangan → GPIO34

Pembagi tegangan: AO → R1 (10k) → titik-tengah → R2 (10k) → GND. Titik-tengah inilah yang ke GPIO34. Ini memotong tegangan jadi setengah (~ aman di bawah 3.3V).

Flame, DHT11, output:

Komponen	Pin komponen	Ke ESP32
Flame sensor	VCC / GND / DO	3.3V / GND / GPIO35
DHT11	VCC / GND / DATA	3.3V / GND / GPIO4

Relay 5V	VCC / GND / IN	5V / GND / GPIO26
Buzzer active	(+) / (-)	GPIO27 / GND
LED indikator	anoda (kaki panjang) / katoda	GPIO25 lewat resistor 220Ω / GND

PENTING

GPIO34 & GPIO35 **input-only** — benar, keduanya memang dipakai sebagai input di sini. Jangan dipakai sebagai output.

Setup datastream Blynk

Buat template “Fire Detector” (lihat bagian *Buat akun & device Blynk*) dengan datastream:

Pin	Nama	Tipe	Widget saran
V0	Gas	Integer (0–4095)	Gauge
V1	Suhu	Double (0–60)	Label / Gauge
V2	Kelembapan	Double (0–100)	Label
V3	Status bahaya	Integer (0–1)	LED

Firmware

Buka `firmware/fire-detector/fire-detector.ino`. Pastikan `secrets.h` sudah diisi dan datastream di atas sudah dibuat. Kode lengkap:

```
// =====
// LAB FIRE DETECTOR (mencakup monitoring kualitas udara)
// PKM IoT STMIK Tazkia x Universitas Pancasila 2026
//
// Fungsi: baca sensor gas/asap (MQ-2), api (flame), suhu & kelembapan
// (DHT11). Kalau gas melewati ambang ATAU api terdeteksi -> bunyikan
// buzzer, nyalakan LED, aktifkan relay (kipas/pompa), kirim status ke
// Blynk. Semua data juga dikirim ke dashboard Blynk tiap 2 detik.
//
// Board : ESP32 DevKit V1
// Toolchain: Arduino IDE
// Library yang perlu di-install (Sketch > Include Library > Manage Libraries):
// - "Blynk" by Volodymyr Shymanskyi
// - "DHT sensor library" by Adafruit (+ "Adafruit Unified Sensor")
// =====

// Blynk perlu 3 #define ini SEBELUM #include <BlynkSimpleEsp32.h>.
// Ketiganya ada di secrets.h (lihat secrets.h.example).
#include "secrets.h"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>

// ----- Pin (lihat tabel wiring di workbook) -----
const int PIN_MQ2_A0 = 34; // MQ-2 analog out -> ADC. GPIO34 input-only.
const int PIN_FLAME = 35; // Flame sensor digital out. GPIO35 input-only.
const int PIN_DHT = 4; // DHT11 data
const int PIN_BUZZER = 27; // Buzzer active (+)
```

```

const int PIN_RELAY    = 26; // Relay IN (kontrol kipas/pompa)
const int PIN_LED      = 25; // LED indikator bahaya

// ----- Ambang batas (kalibrasi saat dry-run) -----
// Nilai gas 0..4095 (12-bit ADC). Di udara bersih biasanya rendah; naik saat
// ada asap/gas. Angka 1500 adalah titik awal -> sesuaikan setelah lihat nilai
// "GAS=" di Serial Monitor pada kondisi udara bersih vs didekatkan asap.
const int AMBANG_GAS = 1500;

// Flame sensor: modul umumnya LOW saat api terdeteksi, HIGH saat aman.
const int FLAME_TERDETEKSI = LOW;

#define DHT_TIPE DHT11
DHT dht(PIN_DHT, DHT_TIPE);

BlynkTimer timer;

// Datastream Blynk:
//  V0 = nilai gas (angka)   V1 = suhu (°C)   V2 = kelembapan (%)
//  V3 = status bahaya (1/0)

void kirimSensor() {
    int gas = analogRead(PIN_MQ2_A0);
    bool api = (digitalRead(PIN_FLAME) == FLAME_TERDETEKSI);

    // DHT bisa gagal baca (kabel longgar / timing). JANGAN kirim angka palsu -
    // laporkan error eksplisit dan lewati pengiriman suhu/kelembapan kali ini.
    float suhu = dht.readTemperature();
    float lembap = dht.readHumidity();
    bool dhtOk = !(isnan(suhu) || isnan(lembap));
    if (!dhtOk) {
        Serial.println("ERROR: DHT11 gagal dibaca (cek kabel data & power 3.3V).");
    }

    bool bahaya = (gas > AMBANG_GAS) || api;

    // Aktuasi lokal
    digitalWrite(PIN_BUZZER, bahaya ? HIGH : LOW);
    digitalWrite(PIN_LED,    bahaya ? HIGH : LOW);
    digitalWrite(PIN_RELAY,  bahaya ? HIGH : LOW);

    // Kirim ke Blynk
    Blynk.virtualWrite(V0, gas);
    if (dhtOk) {
        Blynk.virtualWrite(V1, suhu);
        Blynk.virtualWrite(V2, lembap);
    }
    Blynk.virtualWrite(V3, bahaya ? 1 : 0);

    // Log ke Serial Monitor (Tools > Serial Monitor, 115200 baud)
    Serial.print("GAS="); Serial.print(gas);
    Serial.print(" API="); Serial.print(api ? "YA" : "tidak");
    if (dhtOk) {
        Serial.print(" SUHU="); Serial.print(suhu);
        Serial.print(" LEMBAP="); Serial.print(lembap);
    }
    Serial.print(" -> STATUS="); Serial.println(bahaya ? "BAHAYA" : "aman");
}

void setup() {
    Serial.begin(115200);

    pinMode(PIN_FLAME, INPUT);

```

```

pinMode(PIN_BUZZER, OUTPUT);
pinMode(PIN_RELAY, OUTPUT);
pinMode(PIN_LED, OUTPUT);
digitalWrite(PIN_BUZZER, LOW);
digitalWrite(PIN_RELAY, LOW);
digitalWrite(PIN_LED, LOW);

dht.begin();

// MQ-2 punya elemen pemanas. Pembacaan baru stabil setelah +/- 30-60 detik
// dinyalakan. Wajar kalau nilai gas tinggi lalu turun di menit pertama.
Serial.println("MQ-2 warming up... tunggu ~30 detik sebelum kalibrasi ambang.");

// Blynk.begin() akan connect WiFi + server Blynk. Kalau salah token/SSID,
// ESP32 akan retry terus dan TIDAK lanjut -> cek Serial Monitor.
Blynk.begin(BLYNK_AUTH_TOKEN, WIFI_SSID, WIFI_PASS);

timer.setInterval(2000L, kirimSensor); // baca + kirim tiap 2 detik
}

void loop() {
  Blynk.run();
  timer.run();
}

```

Menjalankan & kalibrasi ambang

1. Upload sketch. Buka Serial Monitor (115200). Tunggu MQ-2 **warm-up** ~ 30-60 detik.
2. Catat nilai `GAS=` di udara bersih (mis. 600-900). Dekatkan asap korek/pemantik (jangan api langsung ke sensor) -> nilai naik tajam.
3. Set `AMBANG_GAS` di kode (baris `const int AMBANG_GAS = ...`) ke nilai di antara kondisi bersih dan berasap. Upload ulang.
4. Uji flame sensor: dekatkan korek (nyala kecil, jarak aman) -> `API=YA`, buzzer + LED + relay aktif.

KALAU ERROR

GAS= selalu 0 atau 4095. 0 = AO tidak nyambung ke GPIO34 / pembagi tegangan salah. 4095 = AO langsung ke 3.3V tanpa pembagi, atau masih warm-up. Cek lagi rangkaian pembagi tegangan.

KALAU ERROR

ERROR: DHT11 gagal dibaca. Kabel DATA longgar, atau DHT diberi 5V (pakai 3.3V). DHT11 juga lambat — wajar sesekali gagal, tapi kalau terus-terusan berarti wiring.

KALAU ERROR

Relay bunyi “klik” terus / kebalik. Sebagian modul relay aktif-LOW. Kalau aktuasi kebalik, tukar logika di kode (`HIGH ⇔ LOW` pada `digitalWrite(PIN_RELAY, ...)`).

KALAU ERROR

Buzzer diam padahal status BAHAYA. Ada buzzer **active** (cukup diberi HIGH, dipakai di sini) dan **passive** (perlu nada/tone). Pastikan pakai buzzer **active**. Cek juga polaritas (+)/(-).

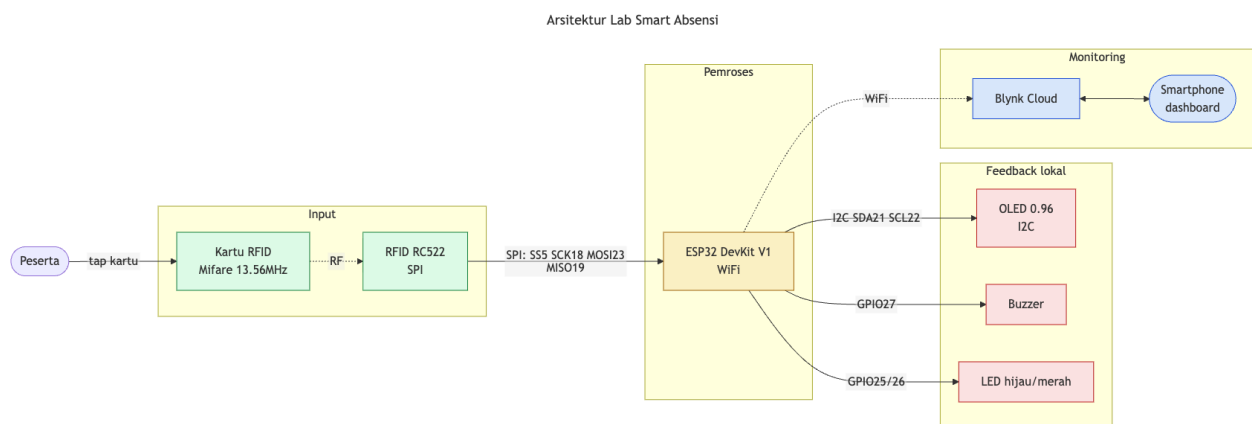
Lab Smart Absensi

Tujuan & konsep

Membangun absensi berbasis kartu RFID: tempel kartu → ESP32 baca nomor unik kartu (**UID**) lewat **RC522**, tampilkan di **OLED**, bunyikan buzzer + LED hijau, lalu kirim UID + nomor urut tap ke dashboard Blynk.

Konsep yang dilatih: komunikasi **SPI** (RC522) dan **I2C** (OLED) — dua protokol standar menghubungkan modul ke mikrokontroler — serta mengirim **data string** (bukan cuma angka) ke cloud.

- **SPI** = protokol cepat 4 kabel (SCK clock, MOSI, MISO, SS). Dipakai RC522.
- **I2C** = protokol 2 kabel (SDA data, SCL clock), tiap perangkat punya alamat. Dipakai OLED (alamat 0x3C).



Daftar komponen (BOM) & estimasi biaya

No	Komponen	Fungsi	Estimasi
1	ESP32 DevKit V1	Mikrokontroler + WiFi	Rp 65.000
2	RFID reader RC522 + kartu	Baca UID kartu (SPI)	Rp 40.000
3	Kartu/keytag RFID tambahan	Identitas peserta	Rp 5.000
4	OLED 0.96" I2C (SSD1306)	Tampilkan UID & status	Rp 35.000
5	Buzzer active	Notifikasi suara	Rp 5.000
6	LED hijau + merah + resistor 220Ω	Indikator OK / gagal	Rp 5.000
7	Breadboard + kabel jumper	Perakitan tanpa solder	Rp 40.000
Total estimasi per kelompok			Rp 195.000

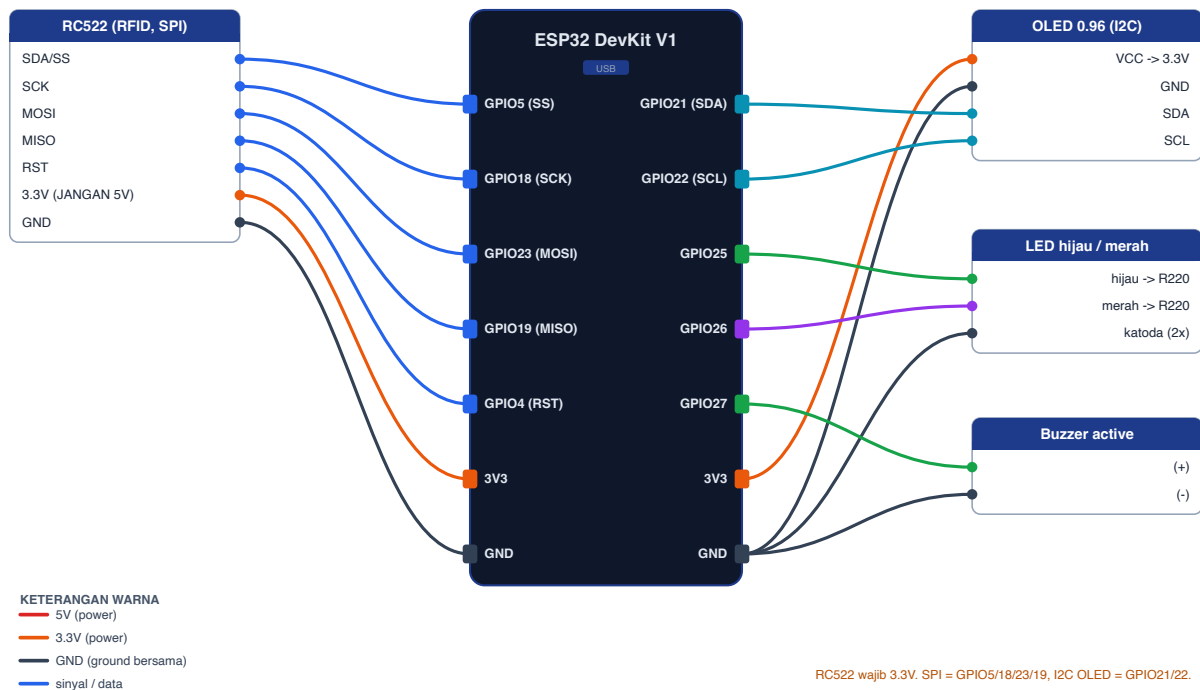
Wiring pin-to-pin

PENTING

RC522 HANYA boleh diberi 3.3V. Memberi 5V akan merusak modul. Ini kesalahan paling umum di lab ini — cek dua kali sebelum colok power.

Wiring Lab Smart Absensi

ESP32 + RC522 (SPI) + OLED (I2C) -> Buzzer + LED



Warna kabel pada diagram = peran (oranye 3.3V, abu GND, biru SPI, cyan I2C). Tabel di bawah adalah rujukan pasti pin-per-pin.

RC522 (SPI):

Pin RC522	Ke ESP32
SDA (SS)	GPIO5
SCK	GPIO18
MOSI	GPIO23
MISO	GPIO19
RST	GPIO4
3.3V	3V3 (JANGAN 5V)
GND	GND
IRQ	tidak dipakai (biarkan kosong)

OLED (I2C) & output:

Komponen	Pin	Ke ESP32
OLED SSD1306	VCC / GND	3.3V / GND
OLED SSD1306	SDA / SCL	GPIO21 / GPIO22
LED hijau	anoda / katoda	GPIO25 lewat 220Ω / GND
LED merah	anoda / katoda	GPIO26 lewat 220Ω / GND
Buzzer active	(+) / (-)	GPIO27 / GND

Setup datastream Blynk

Buat template "Smart Absensi" dengan datastream:

Pin	Nama	Tipe	Widget saran
V0	UID kartu	String	Label
V1	Nomor tap	Integer	Label / Gauge

TIPS

Blynk gratis tidak menyimpan riwayat tabel absensi. Untuk lab ini cukup tampilkan UID & jumlah tap real-time. Mengirim ke database absensi sungguhan (Google Sheets / web server) adalah pengembangan lanjutan — di luar scope lab.

Firmware

Buka `firmware/smart-absensi/smart-absensi.ino`. Pastikan `secrets.h` terisi dan datastream sudah dibuat. Kode lengkap:

```
// =====  
// LAB SMART ABSENSI  
// PKM IoT STMIK Tazkia x Universitas Pancasila 2026  
//  
// Fungsi: baca kartu RFID (RC522) lewat SPI. Saat kartu ditempel, tampilkan  
// UID kartu di OLED, bunyikan buzzer + LED hijau, lalu kirim UID + nomor urut  
// tap ke dashboard Blynk. Kartu tidak terbaca / error -> LED merah.  
//  
// Board : ESP32 DevKit V1  
// Toolchain: Arduino IDE  
// Library yang perlu di-install (Sketch > Include Library > Manage Libraries):  
// - "MFRC522" by GithubCommunity  
// - "Adafruit SSD1306" (+ "Adafruit GFX Library")  
// - "Blynk" by Volodymyr Shymanskyy  
// =====  
  
#include "secrets.h"  
  
#include <WiFi.h>  
#include <BlynkSimpleEsp32.h>  
#include <SPI.h>  
#include <MFRC522.h>  
#include <Wire.h>  
#include <Adafruit_GFX.h>  
#include <Adafruit_SSD1306.h>  
  
// ----- Pin RC522 (SPI) -----  
// PENTING: RC522 HANYA boleh diberi 3.3V. 5V akan merusak modul.  
const int PIN_RC522_SS = 5; // SDA/SS  
const int PIN_RC522_RST = 4; // RST  
// SCK=18, MOSI=23, MISO=19 -> pin SPI default ESP32, tidak perlu di-set manual.  
  
// ----- Pin OLED (I2C) -----  
// SDA=21, SCL=22 -> pin I2C default ESP32.  
const int OLED_W = 128;  
const int OLED_H = 64;  
const int OLED_ADDR = 0x3C; // alamat I2C umum SSD1306 0.96"  
  
// ----- Pin output -----  
const int PIN_LED_HIJAU = 25;
```

```

const int PIN_LED_MERAH = 26;
const int PIN_BUZZER = 27;

MFRC522 rfid(PIN_RC522_SS, PIN_RC522_RST);
Adafruit_SSD1306 oled(OLED_W, OLED_H, &Wire, -1);

int nomorTap = 0;

// Datastream Blynk: V0 = UID kartu (string), V1 = nomor urut tap (angka)

void tampilOled(const String &baris1, const String &baris2) {
  oled.clearDisplay();
  oled.setTextColor(SSD1306_WHITE);
  oled.setTextSize(1);
  oled.setCursor(0, 0);
  oled.println(baris1);
  oled.setTextSize(2);
  oled.setCursor(0, 24);
  oled.println(baris2);
  oled.display();
}

void beep(int ms) {
  digitalWrite(PIN_BUZZER, HIGH);
  delay(ms);
  digitalWrite(PIN_BUZZER, LOW);
}

void setup() {
  Serial.begin(115200);

  pinMode(PIN_LED_HIJAU, OUTPUT);
  pinMode(PIN_LED_MERAH, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);

  // OLED. Kalau gagal (alamat I2C salah / kabel SDA-SCL tertukar) -> stop
  // dengan error eksplisit, jangan lanjut diam-diam.
  if (!oled.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
    Serial.println("ERROR: OLED tidak terdeteksi di 0x3C (cek SDA=21, SCL=22).");
    while (true) { delay(1000); }
  }
  tampilOled("Smart Absensi", "Mulai...");

  // RC522 lewat SPI
  SPI.begin();
  rfid.PCD_Init();

  // Cek RC522 benar-benar terhubung. Versi 0x00 / 0xFF = tidak terdeteksi
  // (biasanya salah kabel atau diberi 5V, bukan 3.3V).
  byte v = rfid.PCD_ReadRegister(MFRC522::VersionReg);
  if (v == 0x00 || v == 0xFF) {
    Serial.println("ERROR: RC522 tidak terdeteksi. Cek wiring SPI & power 3.3V.");
    tampilOled("RC522 error", "Cek wiring");
    digitalWrite(PIN_LED_MERAH, HIGH);
  } else {
    Serial.print("RC522 OK, versi 0x");
    Serial.println(v, HEX);
  }

  Blynk.begin(BLYNK_AUTH_TOKEN, WIFI_SSID, WIFI_PASS);
  tampilOled("Siap.", "Tempel kartu");
}

```

```

void loop() {
  Blynk.run();

  // Tidak ada kartu baru -> keluar cepat.
  if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) {
    return;
  }

  // Susun UID jadi string hex, mis. "A1B2C3D4".
  String uid = "";
  for (byte i = 0; i < rfid.uid.size; i++) {
    if (rfid.uid.uidByte[i] < 0x10) uid += "0";
    uid += String(rfid.uid.uidByte[i], HEX);
  }
  uid.toUpperCase();

  nomorTap++;
  Serial.print("Tap #"); Serial.print(nomorTap);
  Serial.print(" UID="); Serial.println(uid);

  digitalWrite(PIN_LED_HIJAU, HIGH);
  tampilOled("Tap #" + String(nomorTap), uid);
  beep(120);

  Blynk.virtualWrite(V0, uid);
  Blynk.virtualWrite(V1, nomorTap);

  delay(800);
  digitalWrite(PIN_LED_HIJAU, LOW);

  rfid.PICC_HaltA();      // hentikan komunikasi dengan kartu ini
  rfid.PCD_StopCrypto1();
}

```

Menjalankan

1. Upload sketch. Buka Serial Monitor (115200). Harusnya muncul RC522 OK, versi 0x92 (atau 0x91), lalu OLED menampilkan “Siap. Tempel kartu”.
2. Tempel kartu ke RC522. Serial & OLED menampilkan UID (mis. A1B2C3D4), buzzer beep, LED hijau nyala. Dashboard Blynk V0/V1 ikut update.
3. Coba beberapa kartu berbeda — tiap kartu UID-nya unik. Inilah “identitas” yang dipakai untuk absensi.

KALAU ERROR

ERROR: RC522 tidak terdeteksi / versi 0x00 atau 0xFF. Hampir selalu wiring: cek 7 kabel SPI sesuai tabel, dan pastikan power **3.3V** bukan **5V**. Kabel jumper murah sering putus di dalam — coba ganti kabel.

KALAU ERROR

ERROR: OLED tidak terdeteksi di 0x3C. SDA/SCL tertukar (SDA=21, SCL=22), atau alamat I2C modul Anda 0x3D. Ganti OLED_ADDR jadi 0x3D kalau perlu.

KALAU ERROR

Kartu ditempel tapi tidak ada reaksi. Jarak terlalu jauh — tempelkan menyentuh modul. Kartu Mifare 13.56MHz yang didukung; kartu akses gedung 125kHz tidak akan terbaca.

KALAU ERROR

OLED tampil tapi RC522 mati (atau sebaliknya). Power 3.3V ESP32 kadang kurang arus untuk dua modul. Bagikan 3.3V & GND lewat jalur power breadboard, jangan menumpuk jumper di satu lubang pin ESP32.

Lampiran A — Troubleshooting umum

Gejala	Cek ini
Port tidak muncul	Driver USB (CP210x / CH340), kabel data bukan charge-only
Upload “Failed to connect”	Tahan tombol BOOT saat “Connecting...”, pastikan Serial Monitor lain tertutup
Serial Monitor karakter aneh	Baud rate set ke 115200
Macet “Connecting to wifi”	SSID/pass salah, WiFi 5GHz (ESP32 cuma 2.4GHz), captive portal kampus → pakai hotspot HP
Invalid auth token	Salin ulang <code>BLYNK_AUTH_TOKEN</code> dari Device Info
Sensor nilai ngaco	GND belum common — satukan semua GND
Modul mati/panas	Salah tegangan 3.3V vs 5V — RC522 & DHT pakai 3.3V

Lampiran B — Demo Smart Irrigation

Smart Irrigation **bukan** lab hands-on peserta — hanya didemokan trainer di sesi penutup sebagai pembandingan. Arsitekturnya mirip Fire Detector tetapi memakai **sensor kelembapan tanah** (capacitive soil moisture) sebagai input dan **pompa mini via relay** sebagai aktuator: kalau tanah kering melewati ambang, pompa menyala menyiram, lalu mati saat lembap. Konsep yang sama persis dengan yang dipelajari di Lab Fire Detector — baca analog, ambang, aktuasi relay — diterapkan ke kasus berbeda. Prototype dibawa trainer; peserta cukup mengamati dan diskusi.